

# Post-Training: 大模型后训练

## 导论

大模型在完成预训练之后，虽然已经学习了大量语言知识、事实知识和一定的推理能力，但它本质上仍然只是一个根据上下文预测下一个 token 的模型，并不天然具备良好的指令遵循、复杂推理和人类偏好对齐能力。因此，在预训练之后，还需要通过 **Post-Training**，也就是**后训练**，进一步调整模型的行为和能力。

后训练通常包括 SFT、偏好优化、强化学习、知识蒸馏以及数据迭代 等阶段。早期的大模型后训练主要关注“如何让模型更符合人类偏好”，典型路线是 SFT、Reward Model 和 PPO；而随着 DeepSeek-R1、Qwen、Seed 等推理模型的发展，研究重点逐渐转向 Reasoning RL，也就是通过强化学习真正提升模型的数学、代码和复杂推理能力。

因此，现在研究大模型时，除了关注模型架构和预训练规模，后训练方法同样非常重要。不同团队也形成了自己的技术路线，例如 DeepSeek 的 GRPO、Seed 的 DAPO 和 VAPO、Qwen 的 GSPO，以及 GLM、Qwen 等团队进一步研究的 Agent RL。

我们需要理解每一种后训练方法为什么被提出，它解决了前一种方法的什么问题，以及 DeepSeek、Qwen、Seed、GLM 等团队分别做了哪些改进。

## 一、SFT：监督微调

|                                                                                     |                                                                                     |                                                                                     |                                                                                     |                                                                                      |                                                                                       |                                                                                       |
|-------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------|
|  |  |  |  |  |  |  |
| DeepSeek                                                                            | Seed                                                                                | Qwen                                                                                | Gemma                                                                               | GLM                                                                                  | LLaVA                                                                                 | LLaMA                                                                                 |

SFT 是大模型后训练最基础的一步，DeepSeek、Qwen、GLM、Seed、Gemma、Llama 和 LLaVA 都会使用。它的做法就是准备大量“输入—标准回答”数据，然后让模型根据输入预测标准回答中的每一个 token，通过交叉熵损失不断提高正确回答的生成概率。它主要解决的是预训练模型虽然具备大量知识，但并不知道应该按照什么格式回答用户、如何遵循指令以及如何组织推理过程的问题。因此 SFT 本质上是在把一个“续写模型”训练成一个“会按照要求回答问题的模型”，同时也可以通过加入数学推理、代码、长思维链等数据，让模型提前具备一定的任务能力。

## 二、Rejection Sampling：拒绝采样

|                                                                                     |      |                                                                                     |       |     |       |                                                                                       |
|-------------------------------------------------------------------------------------|------|-------------------------------------------------------------------------------------|-------|-----|-------|---------------------------------------------------------------------------------------|
|  |      |  |       |     |       |  |
| DeepSeek                                                                            | Seed | Qwen                                                                                | Gemma | GLM | LLaVA | LLaMA                                                                                 |

Rejection Sampling 本身不是一种新的优化算法，而是一种生成高质量后训练数据的方法，DeepSeek、Qwen 和 Meta Llama 都大量使用。具体做法是让当前模型针对同一个问题生成很多不同回答，然后通过答案验证器、Reward Model、代码执行结果或者人工规则对这些回答进行评分，把错误、低质量或者格式不好的回答删除，只保留质量较高的回答，再把这些高质量数据拿回来继续做 SFT。这样模型实际上是在不断利用自己当前的能力生成新的训练数据，因此随着模型越来越强，能够生成的数

据质量也会越来越高，它是很多模型构建“自我提升数据循环”的方法。

三、 PPO

|          |      |      |       |     |       |                                                                                     |
|----------|------|------|-------|-----|-------|-------------------------------------------------------------------------------------|
|          |      |      |       |     |       |  |
| DeepSeek | Seed | Qwen | Gemma | GLM | LLaVA | LLaMA                                                                               |

PPO 是传统大模型 RLHF 中非常经典的强化学习算法，Meta Llama 的早期后训练以及很多传统 RLHF 系统都使用过。训练时模型作为 Actor 生成回答，然后 Reward Model 对回答进行评分，同时还需要一个 Critic 模型估计当前回答未来可能获得多少奖励，最后根据实际奖励和 Critic 的估计结果计算 Advantage，再更新语言模型，使高奖励回答的概率增加、低奖励回答的概率降低。PPO 的优点是理论和工程体系都比较成熟，但它的问题是需要额外维护一个 Critic，而且对于几十甚至几百亿参数的大模型来说，Critic 会明显增加显存、计算和训练系统复杂度，这也是后来 GRPO 等方法出现的重要原因。

四、 GRPO

|                                                                                    |      |      |       |     |       |       |
|------------------------------------------------------------------------------------|------|------|-------|-----|-------|-------|
|  |      |      |       |     |       |       |
| DeepSeek                                                                           | Seed | Qwen | Gemma | GLM | LLaVA | LLaMA |

GRPO 是 DeepSeek 在 DeepSeekMath 中提出的强化学习方法，后来成为 DeepSeek-R1 系列的重要基础。它和 PPO 最大的区别是取消了单独的 Critic，而是针对同一个问题一次生成一组回答，然后比较这一组回答之间的奖励。例如同一道数学题生成八个答案，其中四个正确、四个错误，那么正确回答相对于这一组的平均表现就是正 Advantage，错误回答就是负 Advantage，模型直接根据这种组内相对表现进行优化。这样做可以在保持强化学习效果的同时减少 Critic 带来的巨大训练成本，因此特别适合大规模语言模型和长思维链推理训练。

五、 R1-Zero Pure RL

|                                                                                     |      |      |       |     |       |       |
|-------------------------------------------------------------------------------------|------|------|-------|-----|-------|-------|
|  |      |      |       |     |       |       |
| DeepSeek                                                                            | Seed | Qwen | Gemma | GLM | LLaVA | LLaMA |

R1-Zero 是 DeepSeek 一个非常有代表性的训练实验，它最大的特点是在预训练模型之后不先进行大规模推理 SFT，而是直接利用 GRPO 做强化学习。训练过程中模型只需要根据数学或者逻辑任务的最终答案是否正确获得奖励，并没有人工告诉模型必须采用什么推理方式，但随着训练进行，模型会逐渐自己产生更长的思维链、自我检查、重新计算和修改推理路径等行为。R1-Zero 说明了一件非常重要的事情，就是很多复杂 reasoning pattern 并不一定需要全部通过人工 SFT 数据教给模型，在足够合适的任务和奖励下，它们可以在强化学习过程中自然形成。不过 Pure RL 的缺点是输出容易出现语言混杂、格式不稳定和可读性差，因此最终的 DeepSeek-R1 又加

入了 Cold Start。

## 六、Cold Start SFT

|                                                                                   |      |      |       |     |       |       |
|-----------------------------------------------------------------------------------|------|------|-------|-----|-------|-------|
|  |      |      |       |     |       |       |
| DeepSeek                                                                          | Seed | Qwen | Gemma | GLM | LLaVA | LLaMA |

Cold Start 是 DeepSeek-R1 在正式进行大规模 reasoning RL 之前加入的一小阶段监督微调。DeepSeek 没有先用非常庞大的 reasoning 数据把模型训练得非常固定，而是只给模型加入一批高质量的长思维链、反思和验证类数据，让模型先形成比较规范的推理格式和基本的 reasoning habit，然后再进行 GRPO 强化学习。这样既保留了 RL 自主探索新推理策略的能力，又解决了 R1-Zero 中出现的输出混乱和可读性差的问题。因此 Cold Start 不是通过大量 SFT 把推理能力全部教完，而是给后续强化学习提供一个更加稳定的起点。

## 七、DeepSeek-R1 多阶段后训练

|                                                                                    |      |      |       |     |       |       |
|------------------------------------------------------------------------------------|------|------|-------|-----|-------|-------|
|  |      |      |       |     |       |       |
| DeepSeek                                                                           | Seed | Qwen | Gemma | GLM | LLaVA | LLaMA |

DeepSeek-R1 的完整后训练不是简单的“SFT 加一次 RL”，而是采用了 Cold Start SFT、Reasoning RL、Rejection Sampling SFT 和 General RL 的多阶段流程。首先通过少量高质量长思维链数据建立基础推理格式，然后使用 GRPO 让模型大量探索数学、代码和逻辑推理能力；接下来让已经强化学习后的模型生成大量新的高质量 reasoning trajectory，通过拒绝采样筛选后，与通用问答、写作等数据一起重新进行 SFT；最后再进行一轮综合强化学习，同时优化 reasoning、helpfulness、instruction following 和安全性。因此 DeepSeek-R1 的一个重要思想就是 RL 不只是最终优化步骤，它还可以用来产生更强的模型，再让这个模型反过来制造下一轮 SFT 数据。

## 八、Scaling-GRPO

|                                                                                     |      |      |       |     |       |       |
|-------------------------------------------------------------------------------------|------|------|-------|-----|-------|-------|
|  |      |      |       |     |       |       |
| DeepSeek                                                                            | Seed | Qwen | Gemma | GLM | LLaVA | LLaMA |

Scaling-GRPO 可以理解为 DeepSeek 针对 GRPO 真正扩展到超大规模训练以后遇到的稳定性问题做的一整套改进。普通 GRPO 在训练规模变大以后会遇到 rollout policy 和当前训练 policy 差距过大、长序列 importance ratio 不稳定、MoE 专家路由不一致以及采样空间不一致等问题，因此 DeepSeek 对 KL 估计、off-policy trajectory 过滤、MoE routing 和 sampling mask 都进行了专门处理。它的目标不是改变 GRPO 最基本的组内相对奖励思想，而是让这种方法在超大模型、MoE、长 CoT 和大量并行 rollout 的情况下仍然能够稳定运行。

## 九、Off-Policy Sequence Masking

|                                                                                   |      |      |       |     |       |       |
|-----------------------------------------------------------------------------------|------|------|-------|-----|-------|-------|
|  |      |      |       |     |       |       |
| DeepSeek                                                                          | Seed | Qwen | Gemma | GLM | LLaVA | LLaMA |

Off-Policy Sequence Masking 是 DeepSeek 在大规模强化学习中解决旧 trajectory 与当前模型差异过大的方法。因为 RL 系统通常会并行生成大量回答，而模型参数又在不断更新，所以某些 trajectory 生成时使用的模型可能已经明显落后于当前模型，如果继续拿这些严重过时的失败样本训练，就可能产生错误或者非常不稳定的梯度。DeepSeek 因此会根据旧策略和当前策略之间的差距识别明显 off-policy 的序列，对其中风险较大的 trajectory 直接进行 mask，不再让它参与当前梯度更新，从而提升大规模 RL 的稳定性。

## 十、Keep Routing

|                                                                                   |      |      |       |     |       |       |
|-----------------------------------------------------------------------------------|------|------|-------|-----|-------|-------|
|  |      |      |       |     |       |       |
| DeepSeek                                                                          | Seed | Qwen | Gemma | GLM | LLaVA | LLaMA |

Keep Routing 是 DeepSeek 针对 MoE 模型强化学习提出的一个非常实用的训练技巧。MoE 模型中的每个 token 会被 Router 分配给不同专家，但是如果 rollout 生成回答时某个 token 使用的是 Expert A 和 Expert B，而真正训练这条 trajectory 时 Router 又把同一个 token 分配到了 Expert C 和 Expert D，那么生成这个行为的参数和最终接受梯度更新的参数就不一致。Keep Routing 的做法就是把 rollout 阶段真实使用的专家路由保存下来，在训练阶段继续使用相同的 routing path，从而保证“哪个专家产生了这个 token，就由哪个专家根据 reward 接受更新”，这对 MoE 强化学习的稳定性非常重要。

## 十一、Keep Sampling Mask

|                                                                                     |      |      |       |     |       |       |
|-------------------------------------------------------------------------------------|------|------|-------|-----|-------|-------|
|  |      |      |       |     |       |       |
| DeepSeek                                                                            | Seed | Qwen | Gemma | GLM | LLaVA | LLaMA |

Keep Sampling Mask 主要解决强化学习 rollout 阶段和训练阶段 token action space 不一致的问题。模型生成回答时通常会使用 Top-p 或 Top-k 等采样方式，因此真正允许采样的 token 只是整个词表中的一部分，但如果训练时计算 policy probability 和 importance ratio 又重新在完整词表上计算，就会造成生成策略和训练策略之间的定义不一致。DeepSeek 的做法是在 rollout 时保存实际使用的 sampling mask，并在训练时继续使用同一个 mask，这样可以让 RL 优化时计算的概率分布和当时真正生成 trajectory 的概率分布保持一致。

## 十二、 Self-Improvement / Self-Evolution

|          |      |                                                                                   |       |     |       |       |
|----------|------|-----------------------------------------------------------------------------------|-------|-----|-------|-------|
|          |      |  |       |     |       |       |
| DeepSeek | Seed | Qwen                                                                              | Gemma | GLM | LLaVA | LLaMA |

Qwen 在 Qwen2.5-Math 等模型中非常强调 Self-Improvement，也就是让模型、训练数据和 Reward Model 一起迭代提升。具体做法是先训练一个当前版本模型，让它针对大量数学问题生成多个候选答案，再利用验证器和 Reward Model 筛选高质量数据，用这些数据继续训练下一版模型；当模型能力提高以后，又重新利用更强的模型生成更难、更高质量的数据，同时 Reward Model 也会重新训练，而不是从头到尾固定一个评价模型。这样就形成 Model、Data 和 Reward Model 共同迭代的闭环，是 Qwen 数学推理后训练中很重要的一条技术路线。

## 十三、 Qwen3 四阶段后训练

|          |      |                                                                                   |       |     |       |       |
|----------|------|-----------------------------------------------------------------------------------|-------|-----|-------|-------|
|          |      |  |       |     |       |       |
| DeepSeek | Seed | Qwen                                                                              | Gemma | GLM | LLaVA | LLaMA |

Qwen3 的旗舰模型采用了比较清晰的四阶段 post-training。第一阶段是 Long-CoT Cold Start，利用数学、代码、逻辑和 STEM 等高质量长推理数据建立基础 reasoning 能力；第二阶段是 Reasoning RL，通过可验证奖励进一步提高复杂推理能力；第三阶段是 Thinking Mode Fusion，把需要长推理的 thinking 数据和可以直接回答的 non-thinking 数据重新混合训练，使一个模型同时具备深度思考和快速回答两种能力；第四阶段再进行 General RL，对指令遵循、人类偏好、通用问答等能力进行整体对齐。它的核心特点是没有单纯追求模型“永远思考得更长”，而是最终希望把 reasoning capability 和普通聊天能力统一到一个模型中。

## 十四、 GSPO

|          |      |                                                                                     |       |     |       |       |
|----------|------|-------------------------------------------------------------------------------------|-------|-----|-------|-------|
|          |      |  |       |     |       |       |
| DeepSeek | Seed | Qwen                                                                                | Gemma | GLM | LLaVA | LLaMA |

GSPO 是 Qwen 针对 GRPO 在长思维链和 MoE 强化学习中稳定性问题提出的 Group Sequence Policy Optimization。GRPO/PPO 中很多优化操作主要发生在 token level，也就是针对每一个 token 单独计算 importance ratio 和 clipping，但是当一条 reasoning trajectory 达到几千甚至几万个 token 时，这种 token-level 的概率波动会不断累积，导致训练噪声和不稳定性增加。GSPO 的思路是把优化单位从单个 token 提高到整个 sequence，在序列层面定义更一致的 policy ratio 和 clipping，使一整条回答在强化学习更新中被作为更完整的行为进行处理，因此尤其适合长 CoT 和 MoE 模型的大规模 RL。

## 十五、 Strong-to-Weak Distillation

|          |      |                                                                                   |       |     |       |       |
|----------|------|-----------------------------------------------------------------------------------|-------|-----|-------|-------|
|          |      |  |       |     |       |       |
| DeepSeek | Seed | Qwen                                                                              | Gemma | GLM | LLaVA | LLaMA |

Strong-to-Weak Distillation 是 Qwen 在小模型后训练中大量采用的一种降成本方法。对于大型旗舰模型，可以投入大量算力完成完整的 Cold Start、Reasoning RL、Thinking Fusion 和 General RL，但是如果每一个 1B、4B、8B 小模型都重新完整跑一遍这个流程，成本会非常高。因此 Qwen 会把已经训练好的强模型作为 Teacher，让小模型学习强模型生成的高质量 reasoning trajectory、输出概率甚至 logits，把大模型通过昂贵强化学习获得的能力直接蒸馏给小模型。这样可以明显减少小模型进行完整 RL 的训练成本，同时保留较好的 reasoning 能力。

## 十六、 AEPO

|          |      |                                                                                   |       |     |       |       |
|----------|------|-----------------------------------------------------------------------------------|-------|-----|-------|-------|
|          |      |  |       |     |       |       |
| DeepSeek | Seed | Qwen                                                                              | Gemma | GLM | LLaVA | LLaMA |

AEPO 是 Qwen 在 Long-Context RL 中使用的 Adaptive Entropy-Controlled Policy Optimization，核心解决的是不同任务需要不同探索强度的问题。强化学习中的 entropy 决定模型生成行为的随机程度，如果 entropy 太低，模型容易一直重复已有推理路径，如果太高又会导致生成过于随机，而长上下文任务的难度差异尤其明显。因此 AEPO 会根据当前任务和训练状态动态调节 entropy，让模型在已经掌握的任务上减少无效探索，在复杂和不稳定的问题上保留更高的探索能力，从而改善长上下文推理强化学习的训练效率和稳定性。

## 十七、 Agent RL

|          |                                                                                     |                                                                                     |       |                                                                                      |       |       |
|----------|-------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------|-------|--------------------------------------------------------------------------------------|-------|-------|
|          |  |  |       |  |       |       |
| DeepSeek | Seed                                                                                | Qwen                                                                                | Gemma | GLM                                                                                  | LLaVA | LLaMA |

Agent RL 与普通问答强化学习最大的区别是训练对象不再只是一次“问题到答案”的映射，而是一整条与环境交互的 trajectory。例如代码 Agent 接到任务以后可能需要查看文件、搜索代码、修改程序、运行测试、观察报错、再次修改，最后才完成任务。训练时模型获得的 reward 通常来自测试是否通过、任务是否完成或者环境最终状态，因此模型学习的不只是最终回答，而是中间每一步工具调用、决策和根据 observation 调整策略的能力。Qwen3-Coder、GLM 系列以及目前很多 Agent 模型都把这类 long-horizon RL 作为后训练的重要方向。

## 十八、 Expert Model Iteration

|          |      |      |       |                                                                                    |       |       |
|----------|------|------|-------|------------------------------------------------------------------------------------|-------|-------|
|          |      |      |       |  |       |       |
| DeepSeek | Seed | Qwen | Gemma | GLM                                                                                | LLaVA | LLaMA |

GLM 的 Expert Model Iteration 思路是先针对不同能力分别训练专家模型，再把这些专家获得的能力融合到统一模型，而不是从一开始要求一个模型同时完成 reasoning、agent 和 general alignment。例如可以先得到一个更强的 Reasoning Expert，再训练一个 Agent Expert，然后利用这些阶段性专家生成数据或者进行蒸馏，把不同专家的能力逐渐汇总到最终模型。这样做的好处是每个阶段的优化目标更加明确，也更容易针对不同能力设计数据和 reward，GLM-4.5 等模型公开的后训练路线中就比较强调这种专家能力逐步迭代和融合的思想。

## 十九、 RLCS

|          |      |      |       |                                                                                    |       |       |
|----------|------|------|-------|------------------------------------------------------------------------------------|-------|-------|
|          |      |      |       |  |       |       |
| DeepSeek | Seed | Qwen | Gemma | GLM                                                                                | LLaVA | LLaMA |

RLCS 是 GLM-V 中使用的 Reinforcement Learning with Curriculum Sampling，也就是把课程学习和强化学习结合起来。它不会在训练开始时随机混合所有难度的多模态任务，而是根据模型当前在不同任务上的表现动态调整采样难度和训练数据，让模型优先训练当前能够获得有效学习信号的样本，然后随着模型能力提高逐渐增加更复杂的 OCR、视频理解、STEM、Grounding、GUIAgent 和长文档任务。它主要解决的是强化学习中数据难度差异过大导致大量样本要么过于简单、要么完全无法获得有效 reward 的问题。

## 二十、 On-Policy Cross-Stage Distillation

|          |      |      |       |                                                                                      |       |       |
|----------|------|------|-------|--------------------------------------------------------------------------------------|-------|-------|
|          |      |      |       |  |       |       |
| DeepSeek | Seed | Qwen | Gemma | GLM                                                                                  | LLaVA | LLaMA |

On-Policy Cross-Stage Distillation 是 GLM 用来解决多阶段强化学习中 catastrophic forgetting 的方法。例如一个模型先完成 Reasoning RL，数学能力已经很好，之后继续进行 Agent RL 时，Agent 能力可能提高，但原来的数学推理能力反而下降。GLM 的方法是保存之前各个阶段的强 checkpoint 作为 Teacher，在新阶段训练当前模型时除了优化新的 RL reward，还利用这些历史 Teacher 的输出概率对当前模型进行蒸馏，让模型在学习 Agent 等新能力的同时尽量保持之前 reasoning 阶段已经获得的能力，因此它本质上是在多个 RL 阶段之间增加一个持续的能力保持约束。

## 二十一、Error Segment Loss Masking

|          |      |      |       |                                                                                    |       |       |
|----------|------|------|-------|------------------------------------------------------------------------------------|-------|-------|
|          |      |      |       |  |       |       |
| DeepSeek | Seed | Qwen | Gemma | GLM                                                                                | LLaVA | LLaMA |

Error Segment Loss Masking 是 GLM 在 Agent trajectory 中处理“模型犯错但后来自己纠正”数据的一种方法。对于一条包含正确操作、错误操作、自我反思和最终修正的轨迹，如果直接删除整条轨迹，就无法训练模型学习错误恢复能力；但如果整条轨迹全部参与监督学习，又会让模型同时学习中间那个错误动作。因此 GLM 的做法是在上下文中保留错误步骤，让模型能够看到错误是如何产生以及后面如何恢复，但在计算训练 loss 时把错误 segment mask 掉，不让模型去模仿错误动作，只学习后续的识别、反思和修正过程。

## 二十二、Asynchronous Agent RL

|          |      |      |       |                                                                                    |       |       |
|----------|------|------|-------|------------------------------------------------------------------------------------|-------|-------|
|          |      |      |       |  |       |       |
| DeepSeek | Seed | Qwen | Gemma | GLM                                                                                | LLaVA | LLaMA |

Asynchronous Agent RL 是 GLM 针对长时间 Agent 环境交互提高训练效率的一种系统设计。普通同步 RL 往往要求一批 rollout 全部生成结束以后才能开始下一轮训练，但是 Agent 任务的执行时间差异很大，有的代码任务几十秒完成，有的可能需要大量工具调用和环境执行，如果全部同步就会产生大量等待时间。异步方法把 rollout generation 和模型 training 解耦，已经完成的 trajectory 可以及时进入训练，而新的 Agent 还可以继续与环境交互，从而减少 GPU 等待环境执行的时间，这对于 coding agent、web agent 和 GUI agent 等长轨迹强化学习非常重要。

## 二十三、DAPO

|          |                                                                                     |      |       |     |       |       |
|----------|-------------------------------------------------------------------------------------|------|-------|-----|-------|-------|
|          |  |      |       |     |       |       |
| DeepSeek | Seed                                                                                | Qwen | Gemma | GLM | LLaVA | LLaMA |

DAPO 是 ByteDance Seed 团队针对 GRPO 在长思维链推理训练中的实际问题提出的一套强化学习改进方法，全称是 Decoupled Clip and Dynamic sAmpling Policy Optimization。它并不是只修改一个公式，而是同时处理 entropy collapse、无效采样、长序列 loss 权重和超长回答 reward 等问题。DAPO 的思想是让模型保留更充分的探索能力，同时只把真正能够产生有效强化学习信号的 prompt 和 trajectory 用于训练，并且重新设计长 CoT 场景中的梯度计算方式，因此它可以理解为 Seed 对 GRPO 进行的一套面向 reasoning RL 的系统性稳定化改造。

## 二十四、Clip-Higher

|          |                                                                                   |      |       |     |       |       |
|----------|-----------------------------------------------------------------------------------|------|-------|-----|-------|-------|
|          |  |      |       |     |       |       |
| DeepSeek | Seed                                                                              | Qwen | Gemma | GLM | LLaVA | LLaMA |

Clip-Higher 是 DAPO 中用来缓解 entropy collapse 的方法。PPO 和 GRPO 都会对旧 policy 的概率比例进行 clipping，防止模型一次更新幅度过大，但 Seed 发现如果正方向的 clipping 上限过小，那些原本概率很低但可能非常有价值的新 token 很难获得足够大的概率提升，长期训练以后模型的生成分布会越来越集中，探索能力不断下降。因此 DAPO 将正向和负向的 clipping 范围进行解耦，并适当放宽概率上升方向的限制，让新的潜在有效 reasoning path 有更大的机会在强化学习过程中被放大。

## 二十五、Dynamic Sampling

|          |                                                                                   |      |       |     |       |       |
|----------|-----------------------------------------------------------------------------------|------|-------|-----|-------|-------|
|          |  |      |       |     |       |       |
| DeepSeek | Seed                                                                              | Qwen | Gemma | GLM | LLaVA | LLaMA |

Dynamic Sampling 是 DAPO 中提高 RL 数据有效利用率的方法。GRPO 依赖同一道题不同 rollout 之间的奖励差异计算相对 Advantage，如果一个问题采样出来的所有回答全部正确，那么这一组样本几乎没有区分度；如果所有回答全部错误，同样很难获得有效的相对学习信号。因此 Seed 会动态过滤这些全对或者全错的 group，并继续补充采样那些模型有时能够答对、有时会答错的问题。这样训练资源更多地集中在模型当前能力边界附近的数据上，可以显著增加每一个 batch 中真正有效的 policy gradient。

## 二十六、Token-Level Policy Gradient Loss

|          |                                                                                     |      |       |     |       |       |
|----------|-------------------------------------------------------------------------------------|------|-------|-----|-------|-------|
|          |  |      |       |     |       |       |
| DeepSeek | Seed                                                                                | Qwen | Gemma | GLM | LLaVA | LLaMA |

Token-Level Policy Gradient Loss 是 DAPO 针对长 CoT 重新设计 loss 聚合方式的方法。传统实现通常先在每一条回答内部对 token loss 求平均，再对不同回答求平均，这样一条 100 token 的短回答和一条 10000 token 的长 reasoning trajectory 在最终 loss 中可能拥有接近的总体权重。DAPO 改成直接把整个 batch 中所有有效 token 的 policy gradient loss 放在一起进行 token-level normalization，使每一个 token 在整体训练中的权重更加一致，从而避免非常长的 reasoning trajectory 因为 sequence-level averaging 而被过度削弱。

二十七、 Overlong Reward Shaping

|          |                                                                                   |      |       |     |       |       |
|----------|-----------------------------------------------------------------------------------|------|-------|-----|-------|-------|
|          |  |      |       |     |       |       |
| DeepSeek | Seed                                                                              | Qwen | Gemma | GLM | LLaVA | LLaMA |

Overlong Reward Shaping 是 Seed 用来处理长思维链被最大生成长度截断的问题。Reasoning RL 中有些模型实际上已经完成了绝大部分正确推理，只是因为达到 maximum generation length，最终答案没有完整输出，如果简单把这种 trajectory 和完全错误的回答一样直接赋予最低 reward，就会产生很大的奖励噪声，甚至让模型错误地降低一些有效长推理路径的概率。DAPO 因此会对接近长度上限的输出进行更加平滑的惩罚或者过滤，减少“因为截断而被错误判成完全失败”的情况。

二十八、 VAPO

|          |                                                                                   |      |       |     |       |       |
|----------|-----------------------------------------------------------------------------------|------|-------|-----|-------|-------|
|          |  |      |       |     |       |       |
| DeepSeek | Seed                                                                              | Qwen | Gemma | GLM | LLaVA | LLaMA |

VAPO 是 Seed 在 reasoning RL 上进一步研究 Value-based 方法得到的一套训练方案，它和 DAPO 的主要区别是重新引入了 Value Model 或 Critic，但专门针对长 CoT 的 credit assignment 问题进行了重新设计。VAPO 会结合 Value Pretraining、不同形式的 GAE、Length-Adaptive GAE、Clip-Higher、Token-Level Loss 和 Group Sampling 等机制，希望让 Critic 能够更加准确地判断长 reasoning trajectory 中不同阶段对最终 reward 的贡献。因此它主要研究的是：在长思维链场景下，如果重新使用 Actor-Critic 架构，怎样让 value estimation 真正稳定有效。

二十九、 Length-Adaptive GAE

|          |                                                                                     |      |       |     |       |       |
|----------|-------------------------------------------------------------------------------------|------|-------|-----|-------|-------|
|          |  |      |       |     |       |       |
| DeepSeek | Seed                                                                                | Qwen | Gemma | GLM | LLaVA | LLaMA |

Length-Adaptive GAE 是 VAPO 中比较有代表性的原创设计。传统 PPO 中 GAE 的参数通常是固定的，但是语言模型的回答长度可能从几十个 token 到几万个 token，相同的 GAE 衰减速度显然无法同时适应短回答和超长 CoT。Seed 因此让 GAE 的参数随着 trajectory 长度动态变化，使短回答更关注局部 reward，而长回答能够把最终 reward 更有效地向前传播到更早的 reasoning token，从而改善长距离 credit assignment，也就是更准确地判断前面某一步推理到底对最终答案产生了多少贡献。

三十、 BOND

|  |  |  |                                                                                     |  |  |  |
|--|--|--|-------------------------------------------------------------------------------------|--|--|--|
|  |  |  |  |  |  |  |
|--|--|--|-------------------------------------------------------------------------------------|--|--|--|

|          |      |      |       |     |       |       |
|----------|------|------|-------|-----|-------|-------|
| DeepSeek | Seed | Qwen | Gemma | GLM | LLaVA | LLaMA |
|----------|------|------|-------|-----|-------|-------|

BOND 是 Google DeepMind 提出的 Best-of-N Distillation，它解决的是 Best-of-N 推理效果好但成本非常高的问题。普通 Best-of-N 会针对同一个问题生成很多回答，再用 Reward Model 选择其中最好的一个，但这意味着实际部署时需要进行 N 次生成。BOND 的思路是把经过 Best-of-N 选择以后得到的优质输出分布重新蒸馏给一个新的 policy，让模型在只生成一次的情况下也尽量接近原本“生成很多次再选最好”的表现。因此可以把 BOND 理解为把一部分 inference-time compute 转化成模型参数中的能力。

### 三十一、WARM

|          |      |      |                                                                                   |     |       |       |
|----------|------|------|-----------------------------------------------------------------------------------|-----|-------|-------|
|          |      |      |  |     |       |       |
| DeepSeek | Seed | Qwen | Gemma                                                                             | GLM | LLaVA | LLaMA |

WARM 是 Google DeepMind 提出的 Weight Averaged Reward Models，用来提高 Reward Model 的稳定性和抗 reward hacking 能力。Reward Model 本身也会受到训练数据噪声、随机初始化和偏好标注差异影响，因此只使用一个 RM 很容易形成错误偏好；传统方法可以训练多个 RM 做 ensemble，但推理成本又会增加。WARM 的做法是独立训练多个 Reward Model，然后直接在参数空间对它们的权重进行平均，得到一个新的单一 Reward Model，从而吸收多个 RM 的共同偏好并降低单一模型的偶然偏差。

### 三十二、WARP

|          |      |      |                                                                                     |     |       |       |
|----------|------|------|-------------------------------------------------------------------------------------|-----|-------|-------|
|          |      |      |  |     |       |       |
| DeepSeek | Seed | Qwen | Gemma                                                                               | GLM | LLaVA | LLaMA |

WARP 是 Google DeepMind 提出的 Weight Averaged Rewarded Policies，核心是在参数空间融合多个经过 reward optimization 的 policy。强化学习通常会提高 reward，但如果训练过度，也可能让模型距离原始 pretrained model 越来越远，导致通用能力、语言质量或者多样性下降。WARP 会对多个独立 RL 得到的 policy 进行权重融合，同时还可以和初始模型参数进行插值，从而在 reward performance 和保持原始模型能力之间取得更好的平衡，因此它研究的本质是 RL 后多个模型 checkpoint 应该怎样组合，而不是只选择最后一个 checkpoint。

### 三十三、Visual Instruction Tuning

|          |      |      |       |     |                                                                                       |       |
|----------|------|------|-------|-----|---------------------------------------------------------------------------------------|-------|
|          |      |      |       |     |  |       |
| DeepSeek | Seed | Qwen | Gemma | GLM | LLaVA                                                                                 | LLaMA |

Visual Instruction Tuning 是 LLaVA 早期非常关键的后训练方法，主要解决多模态指令数据不足的问题。LLaVA 团队会利用已有的图片 caption、目标位置和图像描

述等结构化信息，再调用 GPT-4 自动生成对应的用户问题和高质量回答，从而快速构造大量 image-instruction-response 数据，然后用这些数据训练视觉语言模型，使模型从简单的“看图后生成描述”进一步变成能够按照用户指令进行视觉问答、推理和对话的多模态助手。它的重要意义主要在于证明了可以利用强语言模型大规模合成视觉指令数据。

三十四、Discrepancy-Driven RL Data Selection

|          |      |      |       |     |                                                                                     |       |
|----------|------|------|-------|-----|-------------------------------------------------------------------------------------|-------|
|          |      |      |       |     |  |       |
| DeepSeek | Seed | Qwen | Gemma | GLM | LLaVA                                                                               | LLaMA |

Discrepancy-Driven RL Data Selection 是 LLaVA-OneVision-1.5-RL 中非常有代表性的数据选择方法，它通过比较 Pass@1 和 Pass@N 来判断一个问题是否适合进行强化学习。如果模型一次生成几乎从来答不对，但生成很多次以后偶尔可以找到正确答案，说明模型已经具备完成任务的潜在能力，只是正确推理路径的概率还不够高，这类样本非常适合通过 RL 提高正确 trajectory 的概率；如果 Pass@1 已经很高，问题太简单，如果 Pass@N 仍然接近零，问题对当前模型又太难。因此它本质上是在自动寻找模型当前“会做但还不稳定”的能力边界数据。

三十五、Answer-only

|          |      |      |       |     |                                                                                       |       |
|----------|------|------|-------|-----|---------------------------------------------------------------------------------------|-------|
|          |      |      |       |     |  |       |
| DeepSeek | Seed | Qwen | Gemma | GLM | LLaVA                                                                                 | LLaMA |

LLaVA-OneVision 的 reasoning RL 没有一开始就全部训练长 CoT，而是采用 Answer-only 到 CoT 的 curriculum。第一阶段主要训练模型把视觉问题的最终答案做正确，减少长推理本身带来的噪声，让模型先获得稳定的任务正确率；在此基础上第二阶段再引入更长的 reasoning trajectory，训练模型如何通过多步视觉推理得到结果。这样做可以把“任务正确性”和“推理过程质量”分开逐步优化，相比直接从较弱模型开始进行大规模长思维链强化学习更加稳定。

三十六、Anchor Data

|          |      |      |       |     |                                                                                       |       |
|----------|------|------|-------|-----|---------------------------------------------------------------------------------------|-------|
|          |      |      |       |     |  |       |
| DeepSeek | Seed | Qwen | Gemma | GLM | LLaVA                                                                                 | LLaMA |

Anchor Data 是 LLaVA 在进行多模态 reasoning RL 时防止基础能力退化的一种简单但实用的方法。模型如果长时间只训练复杂 CoT，可能逐渐倾向于所有问题都输出很长的 reasoning，同时一些简单视觉识别和短回答能力反而下降。因此训练 reasoning 数据的同时会继续混入一定比例的普通视觉问答和 answer-only 数据，让这些基础任务持续产生训练信号，相当于在强化复杂 reasoning 能力的同时约束模型不要偏离原有的感知和简洁回答能力。

### 三十七、 SFT + Rejection Sampling + DPO 数据飞轮

|          |      |      |       |     |                                                                                     |       |
|----------|------|------|-------|-----|-------------------------------------------------------------------------------------|-------|
|          |      |      |       |     |  |       |
| DeepSeek | Seed | Qwen | Gemma | GLM | LLaVA                                                                               | LLaMA |

Meta Llama 的后训练特点并不是主要依赖某一种全新的 RL 算法，而是通过大规模数据生成和偏好优化不断迭代模型。一个典型流程是先使用人工和高质量合成数据进行 SFT，然后让当前模型大量生成候选回答，通过 Reward Model 或偏好判断筛选更好的回答形成下一轮 SFT 数据，再构造 chosen 和 rejected 回答对进行 DPO，之后用更新后的模型继续生成质量更高的数据并重复这个过程。因此 Llama 的主要优势更多来自大规模 synthetic data、rejection sampling 和 preference optimization 构成的数据飞轮。

### 三十八、 DPO

|          |      |      |       |     |       |                                                                                     |
|----------|------|------|-------|-----|-------|-------------------------------------------------------------------------------------|
|          |      |      |       |     |       |  |
| DeepSeek | Seed | Qwen | Gemma | GLM | LLaVA | LLaMA                                                                               |

DPO 是 Direct Preference Optimization，它主要用于让模型学习“两个回答中哪个更好”。训练数据通常由一个 prompt、一个 preferred response 和一个 rejected response 组成，DPO 不需要像 PPO 那样先训练 Reward Model 再通过在线强化学习优化模型，而是直接根据偏好数据调整语言模型的概率，使 preferred response 的相对概率提高、rejected response 的概率降低，同时利用 reference model 约束模型不要偏离原模型过远。因此 DPO 的优点是训练结构简单、稳定、成本较低，非常适合通用聊天、人类偏好和 instruction alignment，但它和 reasoning RL 不完全一样，因为它主要利用已经存在的离线偏好数据，而不是让模型在训练过程中持续探索新的解决方案。

### 三十九、 大模型后训练算法基础

我们已经有一个预训练模型  $\pi_{\theta_0}$ ，后训练的本质，是不断改变条件概率分布  $\pi_{\theta}(y|x)$ ，让模型在给定用户输入  $x$  时，把概率质量更多地分配给“我们希望它生成”的回答  $y$ 。

#### LLM 到底在优化什么？

设用户输入为  $x = (x_1, x_2, \dots, x_m)$ ，模型输出为  $y = (y_1, y_2, \dots, y_T)$

由于 Transformer Decoder 是自回归模型，因此完整回答的概率可以分解为

$$\pi_{\theta}(y|x) = \prod_{t=1}^T \pi_{\theta}(y_t|x, y_{<t}), \text{ 其中 } y_{<t} = (y_1, \dots, y_{t-1})$$

取对数  $\log \pi_{\theta}(y|x) = \sum_{t=1}^T \log \pi_{\theta}(y_t|x, y_{<t})$ ，这个公式非常重要。

先从预训练开始。设一段自然语言 Token 为  $z = (z_1, z_2, \dots, z_N)$

语言模型建模  $P_{\theta}(z) = \prod_{t=1}^N P_{\theta}(z_t | z_{<t})$

最大似然估计要求  $\theta^* = \arg \max_{\theta} \sum_{t=1}^N \log P_{\theta}(z_t | z_{<t})$

等价于最小化负对数似然  $\mathcal{L}_{\text{PT}} = - \sum_{t=1}^N \log P_{\theta}(z_t | z_{<t})$

模型最后得到 logits:  $l_t = (l_{t,1}, l_{t,2}, \dots, l_{t,V})$

[假设模型的词表里只有 4 个词: ["苹果", "香蕉", "猫", "电脑"]。

当你给模型输入: "我最喜欢吃的是" 时

**计算 Logits:** 神经网络最后一层输出了一组实数向量:

$$\text{Logits} = [10.5, 8.2, -2.1, 0.4]$$

这些数值可正可负、可大可小，大小代表模型对对应词的“信心”，但它们**不是概率**（加起来不等于 1）。为了把这些得分变成真实的概率，模型会将 Logits 送入

Softmax 函数计算:  $P_j = \frac{\exp(\text{logit}_j)}{\sum_k \exp(\text{logit}_k)}$ ，输出概率分布

$$\text{Probabilities} = [0.91, 0.09, 0.00, 0.00]$$

这时，“苹果”的概率就是 91%，“香蕉”是 9%。]

假设正确 Token 是  $z_t = c$ ，则 one-hot 标签:  $q_{t,j} = \begin{cases} 1, & j = c \\ 0, & j \neq c \end{cases}$

交叉熵  $H(q_t, p_t) = - \sum_{j=1}^V q_{t,j} \log p_{t,j}$ ，因为只有正确 Token 的位置为 1:

$H(q_t, p_t) = - \log p_{t,c}$ ，于是整个序列  $\mathcal{L} = - \sum_t \log p_{t,c}$ ，就是语言模型的 NLL。

**为什么预训练之后还需要后训练?**

关键在于预训练优化的是  $P_{\text{data}}(z_t | z_{<t})$ ，也就是“在互联网文本里，这段文字后面通常出现什么？”

但用户真正想要的是  $P_{\text{good}}(y | x)$ ，也就是“面对我的问题，优秀助手应该回答什么？”，这两者不是同一个分布。

**SFT: 监督微调**

SFT 是整个后训练体系的第一根支柱。

假设数据集为  $\mathcal{D}_{\text{SFT}} = \{(x^{(i)}, y^{(i)})\}_{i=1}^N$ ，其中  $x^{(i)}$  是指令， $y^{(i)}$  是人工、教师模型或者筛选后的高质量回答。

最大化  $\theta_{\text{SFT}} = \arg \max_{\theta} \sum_{(x,y) \in \mathcal{D}} \log \pi_{\theta}(y | x)$

由于  $\log \pi_{\theta}(y | x) = \sum_{t=1}^T \log \pi_{\theta}(y_t | x, y_{<t})$

$$\text{所以 } \theta_{\text{SFT}} = \arg \min_{\theta} - \sum_{(x,y)} \sum_{t=1}^T \log \pi_{\theta}(y_t | x, y_{<t})$$

$$\text{即 } \mathcal{L}_{\text{SFT}} = - \mathbb{E}_{(x,y) \sim \mathcal{D}} [\log \pi_{\theta}(y | x)]$$

为什么 **Prompt** 通常不算 **Loss**?

训练序列可能是

<BOS>

<user>

解释 Transformer

</user>

<assistant>

Transformer 是……

</assistant>

$$\text{定义 mask: } m_t = \begin{cases} 0, & y_t \text{ 属于 Prompt} \\ 1, & y_t \text{ 属于 Assistant Answer} \end{cases}$$

$$\text{实际 Loss: } \mathcal{L}_{\text{SFT}} = - \sum_t m_t \log P_{\theta}(z_t | z_{<t})$$

因此 **User Token: Loss = 0**; **Assistant Token: Loss = CrossEntropy**

目的不是让模型重新记住“用户问题长什么样”，而是训练  $x \rightarrow y$  这个映射。

$$\text{单个训练样本 } \mathcal{L} = - \log \pi_{\theta}(y | x)$$

$$\text{求梯度 } \nabla_{\theta} \mathcal{L} = - \nabla_{\theta} \log \pi_{\theta}(y | x)$$

$$\text{梯度下降 } \theta' = \theta - \eta \nabla_{\theta} \mathcal{L}, \text{ 代入 } \theta' = \theta + \eta \nabla_{\theta} \log \pi_{\theta}(y | x)$$

也就是说: **SFT** 在不断增加训练数据中标准回答的生成概率。

假设  $y_1$  是一个非常好的答案,  $y_2$  是一个勉强可以的答案。SFT 数据只告诉模型是  $y_1$  正确示范, 却没有直接表达  $y_1 > y_2$ , 于是我们希望学习回答之间的**相对偏好**, 这就产生了 **Preference Learning**。

**偏好数据**

构造  $(x, y_w, y_l)$ , 其中  $y_w$  是 Winner,  $y_l$  是 Loser。希望  $y_w \succ y_l$ , 如何把一个“排序关系”转换成可以反向传播的 Loss?

**Reward Model: 奖励模型**

定义一个标量函数  $r_{\phi}(x, y) \in \mathbb{R}$ , 它输入 Prompt + Response, 输出一个实数 Score。

我们不能简单优化  $r_w > r_l$ , 因为这是不可微的比较关系。

$$\text{定义 } P(y_w \succ y_l) = \sigma(r_w - r_l), \text{ 其中 } \sigma(z) = \frac{1}{1 + \exp(-z)}, \text{ 并且 } r_w = r_{\phi}(x, y_w);$$

$r_l = r_{\phi}(x, y_l)$ , 如果  $r_w - r_l \rightarrow +\infty$ , 则  $\sigma(r_w - r_l) \rightarrow 1$ , 因此最大化  $P(y_w \succ y_l)$  等价于最小化  $\mathcal{L}_{\text{RM}} = - \log \sigma(r_{\phi}(x, y_w) - r_{\phi}(x, y_l))$

$$\text{整个数据集 } \mathcal{L}_{\text{RM}} = - \mathbb{E}_{(x, y_w, y_l)} [\log \sigma(r_{\phi}(x, y_w) - r_{\phi}(x, y_l))]$$

现在已经有  $r_\phi(x, y)$ ，那么自然希望  $\max_{\theta} \mathbb{E}_{x \sim \mathcal{D}, y \sim \pi_\theta(\cdot | x)} [r_\phi(x, y)]$ ，即：模型生成回答，然后奖励模型打分，让模型越来越倾向于产生高奖励回答。但这个目标有一个严重的问题。

### Reward Hacking

如果只优化  $\max_{\theta} E[r(x, y)]$ ，模型可能找到 Reward Model 的漏洞。因此模型可能真正回答质量未必提高，但 Reward Score 疯狂提高。所以必须限制模型不要离原始 SFT 模型太远。

定义参考模型  $\pi_{\text{ref}}$ ，一般就是冻结的 SFT 模型。加入 KL 散度  $D_{\text{KL}}(\pi_\theta || \pi_{\text{ref}})$ ，于是完整目标  $J(\theta) = \mathbb{E}_{x, y \sim \pi_\theta} [r(x, y)] - \beta D_{\text{KL}}(\pi_\theta || \pi_{\text{ref}})$

$$\text{对于固定 } x, D_{\text{KL}}(\pi_\theta || \pi_{\text{ref}}) = \sum_y \pi_\theta(y | x) \log \frac{\pi_\theta(y | x)}{\pi_{\text{ref}}(y | x)}$$

于是  $J = \sum_y \pi_\theta(y | x) \left[ r(x, y) - \beta \log \frac{\pi_\theta(y | x)}{\pi_{\text{ref}}(y | x)} \right]$ ，这个式子是理解整个现代偏好训练体系的核心。

### 最优策略

我们直接求最优  $\pi^*(y | x)$ ，最大化：

$$\sum_y \pi(y | x) r(x, y) - \beta \sum_y \pi(y | x) \log \frac{\pi(y | x)}{\pi_{\text{ref}}(y | x)}$$

并满足概率约束  $\sum_y \pi(y | x) = 1$

构造拉格朗日函数：

$$\mathcal{J} = \sum_y \pi(y | x) r(x, y) - \beta \sum_y \pi(y | x) \log \frac{\pi(y | x)}{\pi_{\text{ref}}(y | x)} + \lambda \left( \sum_y \pi(y | x) - 1 \right)$$

$$\text{对 } \pi(y | x) \text{ 求偏导 } \frac{\partial \mathcal{J}}{\partial \pi(y | x)} = r(x, y) - \beta \left[ \log \frac{\pi(y | x)}{\pi_{\text{ref}}(y | x)} + 1 \right] + \lambda$$

$$\text{令其为 0: } r(x, y) - \beta \log \frac{\pi(y | x)}{\pi_{\text{ref}}(y | x)} - \beta + \lambda = 0$$

$$\text{整理 } \log \frac{\pi(y | x)}{\pi_{\text{ref}}(y | x)} = \frac{r(x, y)}{\beta} + C(x)$$

$$\text{因此 } \pi^*(y | x) = \pi_{\text{ref}}(y | x) \exp\left(\frac{r(x, y)}{\beta}\right) \exp(C(x))$$

$$\text{归一化后 } \pi^*(y | x) = \frac{\pi_{\text{ref}}(y | x) \exp(r(x, y)/\beta)}{Z(x)}$$

$$\text{其中 } Z(x) = \sum_y \pi_{\text{ref}}(y | x) \exp(r(x, y)/\beta)$$

RLHF 的最优策略，本质上是在 Reference Model 的概率分布上，用  $\exp(r/\beta)$  重新加权。

问题来了  $y \sim \pi_\theta$  这里的 Token Sampling 是离散的，不能直接计算  $\frac{\partial y}{\partial \theta}$ ，所以需要 Policy Gradient。定义  $J(\theta) = \sum_y \pi_\theta(y|x) R(y)$ ，求梯度  $\nabla_\theta J = \sum_y R(y) \nabla_\theta \pi_\theta(y|x)$

利用恒等式  $\nabla_\theta \pi_\theta = \pi_\theta \nabla_\theta \log \pi_\theta$ ，得到  $\nabla_\theta J = \sum_y \pi_\theta(y|x) R(y) \nabla_\theta \log \pi_\theta(y|x)$ ，于是  $\nabla_\theta J = \mathbb{E}_{y \sim \pi_\theta} [R(y) \nabla_\theta \log \pi_\theta(y|x)]$ ，这就是经典 REINFORCE。

直接用  $R(y)$  方差非常大。加入 baseline  $A(y) = R(y) - b$ 。因为  $E[b \nabla \log \pi_\theta] = 0$ ，所以不会改变期望梯度。常见选择  $b = V_\psi(s)$ ，于是  $A_t = Q(s_t, a_t) - V(s_t)$ ，这就是 Advantage。直观理解：这个动作并不是“好不好”，而是“比这个状态下原本预期的表现好多少”。

### 把 LLM 看成强化学习问题

生成每个 Token 时

状态  $s_t = (x, y_1, \dots, y_{t-1})$

动作  $a_t = y_t$

策略  $\pi_\theta(a_t | s_t)$

状态转移  $s_{t+1} = (s_t, a_t)$

一直生成到 EOS，最终获得 Reward:  $R = r(x, y)$

### PPO

如果直接进行 Policy Gradient，模型参数可能一次更新太大，训练非常容易崩。所以 PPO 的思想是：每一次只允许策略进行有限程度的变化。

旧策略  $\pi_{\theta_{\text{old}}}$ ，新策略  $\pi_\theta$ 。定义  $r_t(\theta) = \frac{\pi_\theta(a_t | s_t)}{\pi_{\theta_{\text{old}}}(a_t | s_t)}$

普通 Policy Gradient 的 surrogate objective 可以写成  $L^{PG} = E[r_t(\theta) A_t]$ ，但是如果  $r_t$  变化过大，训练会不稳定。PPO 做  $\text{clip}(r_t, 1 - \epsilon, 1 + \epsilon)$ ，所以

$$L^{\text{PPO}} = E[\min(r_t A_t, \text{clip}(r_t, 1 - \epsilon, 1 + \epsilon) A_t)]$$

实际训练最大化它。

### DPO

这里进入整个推导中非常漂亮的一部分。刚才已经证明：

$$\pi^*(y|x) = \frac{\pi_{\text{ref}}(y|x) \exp(r(x, y)/\beta)}{Z(x)}$$

两边取对数  $\log \pi^*(y|x) = \log \pi_{\text{ref}}(y|x) + \frac{r(x, y)}{\beta} - \log Z(x)$

于是  $r(x, y) = \beta \log \frac{\pi^*(y|x)}{\pi_{\text{ref}}(y|x)} + \beta \log Z(x)$

注意  $Z(x)$  只和 Prompt 有关。

Bradley-Terry:  $P(y_w \succ y_l | x) = \sigma(r(x, y_w) - r(x, y_l))$

分别写出  $r_w$  与  $r_l$  :

$$r_w = \beta \log \frac{\pi^*(y_w | x)}{\pi_{\text{ref}}(y_w | x)} + \beta \log Z(x); \quad r_l = \beta \log \frac{\pi^*(y_l | x)}{\pi_{\text{ref}}(y_l | x)} + \beta \log Z(x)$$

相减  $r_w - r_l = \beta \left[ \log \frac{\pi^*(y_w | x)}{\pi_{\text{ref}}(y_w | x)} - \log \frac{\pi^*(y_l | x)}{\pi_{\text{ref}}(y_l | x)} \right]$ , 神奇的地方是  $\beta \log Z(x)$  完全消失了。把未知最优策略  $\pi^*$  参数化成  $\pi_\theta$ , 得到:

$$P_\theta(y_w \succ y_l | x) = \sigma \left[ \beta \left( \log \frac{\pi_\theta(y_w | x)}{\pi_{\text{ref}}(y_w | x)} - \log \frac{\pi_\theta(y_l | x)}{\pi_{\text{ref}}(y_l | x)} \right) \right]$$

最大似然就得到 DPO Loss:

$$\mathcal{L}_{\text{DPO}} = -E \left[ \log \sigma \left( \beta \left[ \log \frac{\pi_\theta(y_w | x)}{\pi_{\text{ref}}(y_w | x)} - \log \frac{\pi_\theta(y_l | x)}{\pi_{\text{ref}}(y_l | x)} \right] \right) \right]$$

定义  $\Delta_\theta = \beta \left[ \log \frac{\pi_\theta(y_w | x)}{\pi_{\text{ref}}(y_w | x)} - \log \frac{\pi_\theta(y_l | x)}{\pi_{\text{ref}}(y_l | x)} \right]$ , 则  $L = -\log \sigma(\Delta_\theta)$ 。

求导  $\frac{dL}{d\Delta} = -\frac{\sigma'(\Delta)}{\sigma(\Delta)}$ , 因为  $\sigma'(\Delta) = \sigma(\Delta)(1 - \sigma(\Delta))$

所以  $\frac{dL}{d\Delta} = -(1 - \sigma(\Delta)) = -\sigma(-\Delta)$

进一步  $\nabla_\theta L = -\beta \sigma(-\Delta) [\nabla_\theta \log \pi_\theta(y_w | x) - \nabla_\theta \log \pi_\theta(y_l | x)]$

因此 DPO: **Winner** 相对 **Reference** 的概率提高, **Loser** 相对 **Reference** 的概率降低。

SFT:  $\mathcal{L}_{\text{SFT}} = -\log \pi_\theta(y_w | x)$ , 它只知道把  $y_w$  概率提高。

DPO:  $\mathcal{L}_{\text{DPO}} = f(\pi(y_w), \pi(y_l), \pi_{\text{ref}})$ , 它同时知道  $y_w$  应该更高,  $y_l$  应该更低, 而且要参考原模型。

DPO 很适合“哪个回答更好”的问题, 但是数学、代码等问题有一个非常好的性质: **最终答案往往可以客观验证**。

对于 Prompt  $x$ , 模型生成完整 reasoning trajectory:  $y = (r_1, r_2, \dots, r_T, a)$

其中  $r_1, \dots, r_T$  是推理过程,  $a$  是最终答案。  $V(x, a)$  判断答案, 例如:

$R(x, y) = \begin{cases} 1, & a = a^* \\ 0, & a \neq a^* \end{cases}$ , 优化  $J(\theta) = E_{y \sim \pi_\theta(\cdot | x)} [R(x, y)]$ , 于是模型通过探索逐渐发现哪些 reasoning trajectory 更可能通向正确答案。

假设同一道题一次只生成一个回答, 得到  $R = 0$ , 很难知道到底错得有多严重? 有没有接近正确? 什么策略更好? 因此可以一次采样一组  $y_1, y_2, \dots, y_G \sim \pi_{\theta_{\text{old}}}(\cdot | x)$ , 获得  $R_1, R_2, \dots, R_G$  然后进行相对比较。这就是 GRPO 的重要思想来源。

先计算组内平均奖励  $\mu_R = \frac{1}{G} \sum_{i=1}^G R_i$ , 标准差  $\sigma_R = \sqrt{\frac{1}{G} \sum_{i=1}^G (R_i - \mu_R)^2}$ , 然后构造

$A_i = \frac{R_i - \mu_R}{\sigma_R + \epsilon}$ , 这里不需要额外训练 Value Network。

## GRPO

对第 $i$ 个回答、第 $t$ 个 Token:  $\rho_{i,t}(\theta) = \frac{\pi_{\theta}(y_{i,t} | s_{i,t})}{\pi_{\theta_{\text{old}}}(y_{i,t} | s_{i,t})}$

PPO 风格的目标  $L_{\text{GRPO}} = E[\min(\rho_{i,t} A_i, \text{clip}(\rho_{i,t}, 1 - \epsilon, 1 + \epsilon) A_i)]$

再加入 Reference Model  $\text{KL} J_{\text{GRPO}} = L_{\text{GRPO}} - \beta D_{\text{KL}}(\pi_{\theta} || \pi_{\text{ref}})$ , 所以本质仍然是比组内平均好的轨迹概率上升; 比组内平均差的轨迹概率下降, 同时限制不要漂离 Reference Model 太远。

经典 PPO 通常依赖  $V_{\psi}(s)$  (即 Critic / Value Model), Advantage 为  $A_t = R_t - V_{\psi}(s_t)$ , GRPO 则可以直接利用  $R_i - \text{mean}(R)$  构造相对 Advantage。

因此 GRPO 不必额外维护一个与大模型同量级的 Critic。

这里还有一个很关键的问题。如果推理 3000 Token, 最后才知道答案错了, 那么所有 Token 都收到  $R = 0$ , 这种奖励非常稀疏, 叫 **Outcome Reward**

$$R_{\text{outcome}} = R(x, y_T)$$

**Process Reward** 可以在中间步骤评价  $r_t = r(s_t, a_t)$ , 总 Reward:  $R = \sum_{t=1}^T \gamma^{t-1} r_t$

## Rejection Sampling

假设当前模型  $\pi_{\theta_k}$ , 对于一个问题采样  $y_1, \dots, y_N$ , 得到分数  $R_1, \dots, R_N$ 。选  $y^* = \arg \max_i R_i$ , 或者保留  $R_i > \tau$  的样本。于是得到新数据集  $\mathcal{D}_{k+1} = \{(x, y_i) : R_i > \tau\}$ , 然后再次执行 SFT:  $\theta_{k+1} = \arg \min_{\theta} -E_{(x,y) \sim \mathcal{D}_{k+1}} \log \pi_{\theta}(y | x)$ , 形成

模型  $\pi_k \rightarrow$  生成大量回答  $\rightarrow$  Verifier / Judge  $\rightarrow$

过滤优秀轨迹  $\rightarrow$  构造新训练集  $\rightarrow$  SFT  $\rightarrow$  模型  $\pi_{k+1}$

这实际上是一种迭代分布提升原来的数据分布  $q_k(y | x)$ , 经过 Reward 筛选  $q_{k+1}(y | x) \propto q_k(y | x) I(R(y) > \tau)$ , 其中  $I(\cdot)$  是指示函数。模型再去拟合  $q_{k+1}$ , 于是概率质量逐渐集中在高质量区域。

## 蒸馏

假设 Teacher  $\pi_T$ ; Student  $\pi_S$ 。最简单的是 Teacher 生成  $y^T$ , 然后 Student 做 SFT:  $\mathcal{L}_{\text{distill}} = -E[\log \pi_S(y^T | x)]$ , 这实际上叫 Sequence-level Distillation。

如果能够获得 Teacher 的完整 Token 分布  $p_T(v | s_t)$ , Student 为  $p_S(v | s_t)$ , 可以最小化  $D_{\text{KL}}(p_T || p_S)$ 。展开  $\mathcal{L}_{\text{KD}} = \sum_v p_T(v | s_t) \log \frac{p_T(v | s_t)}{p_S(v | s_t)}$ , 由于第一部分与 Student 无关 ( $\sum_v p_T(v) \log p_T(v)$ ), 因此等价于最小化  $\mathcal{L}_{\text{KD}} = - \sum_v p_T(v | s_t) \log p_S(v | s_t)$ , 这和 one-hot SFT 的区别非常重要, 普通 SFT 只有正确 Token 有信息。Teacher Distribution: A: 0.65, B: 0.20, C: 0.10, D: 0.05 (包含了更丰富的“软知识”)。

## 现代 Reasoning Distillation

Teacher 不只是生成最终答案  $a$ , 而是生成  $y = (r_1, r_2, \dots, r_T, a)$ 。

Student 学  $\mathcal{L} = - \sum_t \log \pi_S(y_t^T | x, y_{<t}^T)$ , 于是 Student 不只是学习“答案是什么”, 还学习“一种能够到达答案的输出轨迹分布”。

### 安全训练

可以把模型目标看成多目标优化  $R = \alpha R_{\text{helpful}} + \beta R_{\text{correct}} + \gamma R_{\text{safety}} + \delta R_{\text{format}}$   
但更严谨的方式是把安全看成约束。例如  $\max_{\theta} E[R_{\text{helpful}}]$ , 满足  $E[C_{\text{risk}}] \leq \epsilon$

构造拉格朗日函数  $J(\theta, \lambda) = E[R_{\text{helpful}}] - \lambda(E[C_{\text{risk}}] - \epsilon)$

于是模型同时学习 **提高帮助性 + 控制危险行为概率**。

### Tool / Agent Training

当模型拥有工具之后, Action 不再只是 Token。可以定义:

$$a_t \in \{\text{Token, Search, Calculator, Code, Browser, ...}\}$$

状态  $s_t = (x, a_{<t}, o_{<t})$ , 其中  $o_t$  表示工具返回 Observation。

这已经是标准的序列决策问题  $\tau = (s_1, a_1, s_2, a_2, \dots, s_T, a_T)$

目标  $J(\theta) = E_{\tau \sim \pi_{\theta}} \left[ \sum_{t=1}^T \gamma^{t-1} r_t \right]$ , 所以 Agent Training 从数学上仍然属于强化学习框架。